

A UTP Refinement Model of the STeC Language

Yuanrui Zhang^{*†} Yujing Ma^{*} Yixiang Chen^{*‡}

^{*}School of Computer Science and Software Engineering,

MoE Engineering Center for Software/Hardware Co-design Technology and Application,
East China Normal University, Shanghai 200062, China.

[†]Email: zhangyrmath@126.com

[‡]Corresponding Author. Email: yxchen@sei.ecnu.edu.cn

Abstract—The Internet of Things (IoT) and Cyber-Physical Systems (CPS) are new trends of real-time systems in the area of information technology. The consistency between time and location is a basic feature of IoT and CPS. Such spatial-temporal consistency means that an agent arrives at the required location at the required time and finishes the required tasks within the required time. In order to describe the requirement of spatial and temporal consistency of cyber-physical systems, a specification language called STeC was proposed by Chen in 2012 [1]. This paper will focus on formalising the refinement of STeC programs using the UTP methods. For the STeC language, algebraic refinement, denotational refinement and operational refinement are introduced based on He's presentation on a new roadmap of linking theories of programming. The equivalence among these three kinds of refinement is given.

Index Terms—Program algebra, Spatio-Temporal, Semantics refinement

I. INTRODUCTION

The aim of formal description methods is to give unambiguous definitions of languages, mathematically describing and reasoning about systems. Hoare provides the method for proving that formal description methods at different approaches of formalisation are consistent relative to each other in [2].

Hoare and He proposed the Unifying Theories of Programming [3] (UTP, shortly), which is a milestone for the process of building coherent and comprehensive theories for the unification of languages. In UTP, denotational semantics is given first, whereas algebraic semantics also plays an important role in formal description methods. [4] expresses the view that learning from the set of algebraic laws that links with other abstraction theories is the best possible way of understanding programming paradigms. [5] proposes an algebraic approach to describe and reason about logic programs.

To establish the comparability between two programs, it is necessary to adopt the concept of refinement [6]. Taking Communicating Sequential Processes [7] (CSP, shortly) as an example, a program P is refined by a program Q , if Q satisfies any specification of P . [7] and [8] define $P \sqsubseteq Q$ to capture that Q is more predictable and controllable than P , while Q must be able to guarantee everything done by P .

Recently, He's presentation on a new roadmap for linking theories of programming revisits the approach on linking three types of formalisms [9]. Unlike the top-down approach with denotational models given first and bottom-up approach with operational models given first, the algebra of the program is

defined first. According to full abstraction with denotational models and consistency with the step relation of operational models, the algebra of programs can be used to build up denotational and operational models.

Chen introduces the Spatio-Temporal Consistence Language in 2012 [1], denoted as STeC, to specify real-time systems. STeC looks like an extension of timed CSP [10] and timed CCS [11] that includes actions and communications. The meaning of spatio-temporal consistency is that an agent arrives at the required location at the required time and finishes the required task(s) within the required time. In the previous work, Wu revisits the operational semantics of STeC on preconditions of the choice rule in [12], there the full abstraction is analyzed between operational and denotational semantics except for the parallel operator ' $\parallel$ '. Zhang defines the theoretical transformation from spatio-temporal models to timed automata [13]. Yang devises a domain Spatio-Temporal language for the application domain of space-air-ground integrated networks in [14]. He proposes PSTeC by adding probabilities to the STeC language in [15].

In this paper, we build the semantical framework of the STeC language, and introduce algebraic refinement, denotational refinement and operational refinement based on He's presentation [9]. We define the algebraic semantics of the STeC language after introducing the refinement of data states. We prove the equivalence of refinements of three different semantics. Once the algebra of a STeC program is given, the denotational semantics and the operational semantics of the STeC program can be built up from the algebra. Thus we get observable properties and individual steps of possible behaviours only by equations of STeC programs.

The rest of this paper is organized as follows: Section II introduces some preliminaries about the STeC language. In Section III, we introduce data states of STeC programs and explore their basic algebraic properties. The algebraic refinement relations between data states form the theoretical starting point of building the semantic theories of STeC is in the remaining sections. From the last part of Section III to Section VI we respectively construct the algebraic semantics, the denotational semantics and the operational semantics of the STeC language based on the algebraic properties of data states. In Section VII, we present the main result of our proposed framework – the equivalence of the three semantics. This result guarantees that the method we use to build the semantics of STeC is complete.

II. STeC LANGUAGE

We provide a simple preliminary for STeC and two examples modelled by STeC.

A. Syntax of STeC

The most recent version of STeC is given below (readers can find more details in [1]):

$$\begin{aligned}
A &::= \text{Skip}@(\ell, t) \mid \text{Stop}@(\ell, t) \mid \\
&\quad \text{Send}(m)@(\ell, t) \mid \text{Get}(m)@(\ell, t) \mid \\
&\quad \text{Sens}(m, \delta)@(\ell, t) \mid \text{Comp}(m, m', \delta)@(\ell, t) \\
B &::= \text{Act}(\ell', \delta)@(\ell, t) \mid \text{Stat}(\delta)@(\ell, t) \\
P &::= A \mid B \mid B \rightarrow P \mid P \square P \mid P \parallel P \mid P; P
\end{aligned}$$

In STeC, we use Σ to denote the infinite set of action names. Processes communicate with each other through sending/receiving messages. $\mathcal{M}$ denotes an infinite set of message names, ranged over by m . $\mathcal{L}$ is an infinite set of location names, ranged over by ℓ . $\mathbb{R}^+$ represents non-negative real time values, ranged over by t . The set of all processes is denoted as $\mathbb{P}$, ranged over by alphabets P, Q, R .

The formulas in STeC are divided into atomic formulas and compositional ones:

Atomic formulas. There are two types of atomic processes, types A and B , as shown above. Action *Skip* successfully terminates without any change to the system after its execution. Unlike traditional process algebras like CSP or CCS where action *Stop* means deadlock, *Stop* in STeC means the failure of termination – ‘the conditions of both divergence and deadlock. Action *Skip*@(ℓ, t), *Stop*@(ℓ, t) bounded with a space-time condition ‘@(ℓ, t)’ means that this action occurs at required time $t \in \mathbb{R}^+$ and location $\ell \in \mathcal{L}$.

Communications between processes in STeC are one-to-one communications. *Send*(m)@(ℓ, t) and *Get*(m)@(ℓ, t) are communication actions between agents. *Send*(m)@(ℓ, t) (resp. *Get*(m)@(ℓ, t)) means that an agent sends (resp. receives) a message (named $m \in \mathcal{M}$) at (ℓ, t).

Action *Sens*(m, δ)@(ℓ, t) is used in the situations when it takes time for an agent to receive a message (compared with *Get*(m)@(ℓ, t)). The agent takes δ units of time to receive message m at (ℓ, t).

Action *Comp*(m, m', δ)@(ℓ, t) describes that an agent at (ℓ, t) receives a message m and produces a new message m' for δ units of time.

Action *Act*(ℓ', δ)@(ℓ, t) is a movement from location ℓ to ℓ' . It means that an agent performs action $\text{Act} \in \Sigma$ at (ℓ, t), taking δ units of time ($\delta \in \mathbb{R}^+$), and arrives at location ℓ' . *Stat*(δ)@(ℓ, t) is a state in which an agent stays. It means that the agent remains in state $\text{Stat} \in \Sigma$ at (ℓ, t), for δ units of time.

In STeC, we define a boolean function $\mathcal{T}(B) = \{\text{true}, \text{false}\}$ assigning each atomic process of type B to a boolean value. $\mathcal{T}(B) = \text{true}$ means that an action (of type B) is successfully executed. For example, $\mathcal{T}(\text{Act}(\ell', \delta)@(\ell, t)) = \text{true}$ means that action *Act* is executed at location ℓ and time t , taking δ units of time and ends at ℓ' . Function

$\ell(B) \in \mathcal{L}$ returns the location value of an action of type B , e.g. $\ell(\text{Act}(\ell', \delta)@(\ell, t)) = \ell$. Function $\tau(B) \in \mathbb{R}^+$ returns the execution time of an action of type B , e.g. $\tau(\text{Act}(\ell', \delta)@(\ell, t)) = t$. Readers can see more formal definitions of these functions in [1].

Compositional formulas. $B \rightarrow P$ stands for the ‘if-then’ statement. Guard B is an action of type B . It means that if guard B meets $\mathcal{T}(B) = \text{true}$, then process P proceeds. If B satisfies $\mathcal{T}(B) = \text{false}$, *Stop* proceeds. $P \square Q$ is the choice operator, meaning that the agent behaves either like P or Q but not both. $P \parallel Q$ is the parallel operator, meaning P and Q run in parallel. $P; Q$ is the sequential operator, it means that after P successfully terminates, Q proceeds.

B. Example of STeC

We give some simple examples to model real-time systems using STeC.

1) *A Sensor*: Consider a sensor with the ability to sense data from the environment, to compute the acquired data and to send the result. Using STeC we model this system as follows:

$$\begin{aligned}
\text{Sensor} &= \text{idle}(60s)@(\ell, t); \text{Sens}(m_0, 1s)@(\ell, t + 60s); \\
&\quad \text{Comp}(m_0, m_1, 3s)@(\ell, t + 61s); \\
&\quad \text{Send}(m_1)@(\ell, t + 64s); \text{idle}(10s)@(\ell, t + 64s); \\
&\quad \text{Get}(m_2)@(\ell, t + 74s).
\end{aligned}$$

The behaviour of the sensor is: the sensor keeps idling for 60 seconds at (ℓ, t), and senses data m_0 , taking 1 second; then it computes m_1 from m_0 for 3 seconds, sending data m_1 ; after that it sends m_1 (to another agent), and then waits for 10 seconds to receive the message m_2 .

2) *Two Cars*: Another example considers two cars running on the Huning Expressway from the city Shanghai, through the city Suzhou, Wuxi, Changzhou and Zhenjiang, to the city Nanjing at the same time. The behaviour of two car agents *Car*₁ and *Car*₂ are modelled using STeC as follows:

$$\begin{aligned}
\text{Car}_1 &= \text{Run}(\text{Nanjing}, 4h)@(\text{Shanghai}, 12 : 00) \\
&\quad \parallel \text{Send}(\text{Suzhou})@(\text{Suzhou}, 13 : 00) \\
&\quad \parallel \text{Send}(\text{Wuxi})@(\text{Wuxi}, 13 : 30) \\
&\quad \parallel \text{Send}(\text{Nanjing})@(\text{Nanjing}, 16 : 00) \\
\text{Car}_2 &= \text{Run}(\text{Nanjing}, 4h)@(\text{Shanghai}, 12 : 00) \\
&\quad \parallel \text{Send}(\text{Suzhou})@(\text{Suzhou}, 13 : 00) \\
&\quad \parallel \text{Send}(\text{Changzhou}, 14 : 30) \\
&\quad \parallel \text{Send}(\text{Nanjing})@(\text{Nanjing}, 16 : 00)
\end{aligned}$$

Agents *Car*₁ and *Car*₂ both run from Shanghai to Nanjing, starting at noon (12:00), and arriving at afternoon (16:00). They send message *Nanjing* to the control center when they reach location *Nanjing* on time. Also they both send messages *Suzhou* at location *Suzhou* to the control center. In addition, *Car*₁ sends message *Wuxi* to the control center at 13:30 when arriving *Wuxi*, while *Car*₂ sends message *Changzhou* to the control center when it reaches *Changzhou* at 14:30.

III. DATA STATE DOMAIN

The execution of a STeC program consists of individual steps from one state to another. The state consists of three components: 1) The current location. 2) The current time. 3) The collection of messages.

Definition 1 (Data State). *The data state of a STeC program is a triple (a, u, σ) , where $a \in \mathcal{L}$ is location, $u \in \mathbb{R}^+$ is time, $\sigma \in 2^{\mathcal{M}}$ is a subset of messages received by an agent till now. The set of all data states is denoted as $ST = \{(a, u, \sigma) \mid a \in \mathcal{L}, u \in \mathbb{R}^+, \sigma \in 2^{\mathcal{M}}\}$. $ST^* = \{S \mid S \subseteq_f ST\}$ is the set of all finite subsets of ST . ST^* contains the empty set $\emptyset$, we denote it as $\top$. Let $\perp = ST$ be the total set. Let $ST_{\perp}^* = ST^* \cup \{\perp\}$.*

We define the set $ST_{\perp}^*$ – all finite sets of data states ST^* including $\perp$ because $\perp$ is clearly not finite. $\perp$ denotes an important notion of STeC process – the failure of termination. Since $\perp = ST$, $\perp$ represents the most non-deterministic data state. $\top = \emptyset$ is the empty set, representing the most deterministic data state.

To analyze the algebraic equivalence of STeC formulas, we first give the algebraic refinement relationship between sets of data states.

Definition 2 (Algebraic Refinement of Data State). *The algebraic refinement relationship $\sqsubseteq_A$ introduced by He in [9], is defined as a partial order relation on ST^* that satisfies:*

- 1) $\forall S \in ST^* \cdot (\perp \sqsubseteq_A S)$.
- 2) $\forall S, T \in ST^* \cdot (S \sqsubseteq_A T \text{ iff } T \subseteq S)$.

Intuitively, for any data states S and T with $S \sqsubseteq_A T$, we can think T as a ‘more deterministic data state’ than S .

Proposition 1. $\sqsubseteq_A$ is reflexive and transitive.

Proof. The result immediately follows from the reflexivity and transitivity of $\subseteq$. $\square$

With $\sqsubseteq_A$, we can induce the algebraic equivalence between data states.

Definition 3 (Algebraic Equivalence). *For any $S, T \in ST_{\perp}^*$, $S =_A T$ iff $S \sqsubseteq_A T$ and $T \sqsubseteq_A S$.*

In the analysis of the algebraic semantics of STeC later in Section III-A, we need to define two operators between data states. Combining processes using the operator $\square$ or $\parallel$ increases the nondeterminacy of program results. We have a slightly different view about the specification language STeC. The operator $\square$ in STeC decreases the nondeterminacy, while the operator $\parallel$ increases the nondeterminacy. We therefore introduce two operators $\bigvee$ and $\bigwedge$, which are used to distinguish the final data states of $P \square Q$ and $P \parallel Q$.

Definition 4 ($\bigvee$ and $\bigwedge$). $\forall S, T \in ST_{\perp}^*$,

- 1) $S \bigvee T = \begin{cases} \emptyset & \text{if } S = \emptyset \vee T = \emptyset \\ S' & \text{else} \end{cases}$,
where $S' = \{(a, u, \sigma_1 \cap \sigma_2) \mid (a, u, \sigma_1) \in S \wedge (a, u, \sigma_2) \in T\}$.
- 2) $S \bigwedge T = S \cup T$.

Operator $\bigwedge$ is the greatest upper bound of any two sets of data states. The partial order set $(ST_{\perp}^*, \sqsubseteq_A)$ is in fact a $\bigwedge$ semi-lattice, as stated in the next proposition.

Proposition 2 (Semi-lattice $(ST_{\perp}^*, \sqsubseteq_A)$). *Poset set $(ST_{\perp}^*, \sqsubseteq_A)$ is a $\bigwedge$ semi-lattice with $\perp$ of its bottom.*

Proof. For any $S, T \in ST_{\perp}^*$, we prove that $S \bigwedge T$ is their greatest lower bound. In fact, since $S \bigwedge T = S \cup T$, so $S \subseteq S \cup T$, thus by definition of $\sqsubseteq_A$, we have: $S \bigwedge T \sqsubseteq_A S$.

Similar for $S \bigwedge T \sqsubseteq_A T$. $\square$

We have deduced several basic laws for operator $\bigvee$ and $\bigwedge$.

Proposition 3. $\forall S, T \in ST_{\perp}^*$,

- (1) $\perp \bigwedge S =_A \perp$.
- (2) $\top \bigvee S =_A \top$, $\top \bigwedge S =_A S$.
- (3) $S \bigwedge T \sqsubseteq_A S$, $S \bigwedge T \sqsubseteq_A T$.

Proof. (1) Since $\perp$ is the smallest element in $(ST_{\perp}^*, \sqsubseteq_A)$, so $\perp \sqsubseteq_A \perp \bigwedge S$. On the other hand, $\perp \bigwedge S$ is the greatest upper bound of $\perp$ and S , so $\perp \bigwedge S \sqsubseteq_A \perp$.

(2) $\top = \emptyset$, we have $\top \bigvee S = \emptyset = \top$ by Definition 4. For the same reason, $\top \bigwedge S = \top \cup S = S$.

(3) is obvious since $(ST_{\perp}^*, \sqsubseteq_A)$ is a $\bigwedge$ semi-lattice. $\square$

He introduces the notion of testing of general languages, to test the status of programs and to examine the algebraic properties in [9]. Based on He’s idea, we define the notion of testing of the STeC language, called Testing, so that we can record the algebra of STeC formulas.

Definition 5 (Testing). *The testing of a STeC program P given an initial data state (a, u, σ) is given by $(a, u, \sigma)^\circ P$. It ends with a finite set of data states, called the final data state, or fails to terminate:*

$$(a, u, \sigma)^\circ P =_{df} \begin{cases} \perp \\ T \end{cases}$$

where $T \in ST^*$ is a finite subset of ST .

We extend the testing of STeC programs with a single data state to the one with a set of data states. It is necessary for the algebraic semantics of STeC given next.

Definition 6 (Testing with a set of data states). *The testing of a STeC program P given a set of initial data states $S \in ST_{\perp}^*$ is given by $S^\circ P$. It ends with a set of data states, called the final data state, defined as:*

$$S^\circ P =_{df} \begin{cases} \perp & \text{if } S = \perp \\ \bigwedge_{(a, u, \sigma) \in S} (a, u, \sigma)^\circ P & \text{else} \end{cases}$$

We have the following propositions concerning $\top$.

Proposition 4. $\top^\circ P = \top$

Proof. Since $\top = \emptyset$. $\square$

A. Algebraic Semantics

After the definition of the testing of STeC programs, we now need to give the testing of every STeC formula.

- 1) $(a, u, \sigma)^\circ \text{Skip}@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \{(l, t, \sigma)\} & \text{else} \end{cases}$

- 2) $(a, u, \sigma) \circ \text{Stop}@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \perp & \text{else} \end{cases}$
- 3) $(a, u, \sigma) \circ \text{Send}(m)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \{(l, t, \{m\} \cup \sigma)\} & \text{else} \end{cases}$
- 4) $(a, u, \sigma) \circ \text{Get}(m)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \top & \text{if } m \notin \sigma \\ \{(l, t, \sigma \setminus \{m\})\} & \text{else} \end{cases}$
- 5) $(a, u, \sigma) \circ \text{Act}(l', \delta)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \{(l', t + \delta, \sigma)\} & \text{else} \end{cases}$
- 6) $(a, u, \sigma) \circ \text{Stat}(\delta)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \{(l, t + \delta, \sigma)\} & \text{else} \end{cases}$
- 7) $(a, u, \sigma) \circ \text{Sens}(m, \delta)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \{(l, t + \delta, \{m\} \cup \sigma)\} & \text{else} \end{cases}$
- 8) $(a, u, \sigma) \circ \text{Comp}(m, m', \delta)@ (l, t) = \begin{cases} \top & \text{if } (a, u) \neq (l, t) \\ \top & \text{if } m \notin \sigma \\ \{(l, t + \delta, \sigma[m'/m])\} & \text{else} \end{cases}$
- 9) $(a, u, \sigma) \circ (B \rightarrow P) = \begin{cases} ((a, u, \sigma) \circ B) \circ P & \text{if } \mathcal{T}(B) = \text{true} \\ \perp & \text{else} \end{cases}$
- 10) $(a, u, \sigma) \circ (P; Q) = \begin{cases} \top & \text{if } (a, u, \sigma) \circ P = \top \\ ((a, u, \sigma) \circ P) \circ Q & \text{else} \end{cases}$
- 11) $(a, u, \sigma) \circ (P \parallel Q) = (a, u, \sigma) \circ P \wedge (a, u, \sigma) \circ Q$
- 12) $(a, u, \sigma) \circ (P \sqcap Q) = (a, u, \sigma) \circ P \vee (a, u, \sigma) \circ Q$

When the space-time pair (a, u) does not coincide with current space-time condition (l, t) (i.e. $(a, u) \neq (l, t)$), the process is locked and produces nothing. If the space-time pair meets, it returns $\perp$, the worst data state.

Action $\text{Get}(m)@ (l, t)$ takes away a message m from the agent's storage σ . While action $\text{Send}(m)@ (l, t)$ sends a message m in σ . Their testings return corresponding data states. Action $\text{Sens}(m, \delta)@ (l, t)$ senses a message m and adds it into σ after δ units of time. Action $\text{Comp}(m, m', \delta)@ (l, t)$ replaces message m with m' after δ units of time. For the testing of $B \rightarrow P$, if $\mathcal{T}(B) = \text{true}$, we can regard the output of the testing of $(a, u, \sigma) \circ B$ as the initial data state of the testing $((a, u, \sigma) \circ B) \circ P$; otherwise $B \rightarrow P$ diverges. In $(a, u, \sigma) \circ (P; Q)$, we first compute $(a, u, \sigma) \circ P$, and if it is not the empty set, we then compute $(a', u', \sigma') \circ Q$ for every $(a', u', \sigma') \in (a, u, \sigma) \circ P$. The testing of $(a, u, \sigma) \circ (P \parallel Q)$ returns all the possible results of the testing of $(a, u, \sigma) \circ P$ and $(a, u, \sigma) \circ Q$. $(a, u, \sigma) \circ (P \sqcap Q)$ only captures the common space-time points of P and Q . Messages of $(a, u, \sigma) \circ (P \sqcap Q)$ consist of the common messages from P and Q .

B. An Example of Data States

For $P \sqcap Q$ the final space-time points of P and Q must be the same, in case the subsequent process has the nondeterministic initial space-time point after P or Q is selected. Taking the second example in Section II-B, the final data states of Car_1 are

$$\begin{aligned} & (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_1 \\ &= \{(\text{Shanghai}, 12 : 00, \emptyset), (\text{Suzhou}, 13 : 00, \{\text{Suzhou}\}), \\ & \quad (\text{Wuxi}, 13 : 30, \{\text{Suzhou}, \text{Wuxi}\}), \\ & \quad (\text{Nanjing}, 16 : 00, \{\text{Suzhou}, \text{Wuxi}, \text{Nanjing}\})\}. \end{aligned}$$

And the final data states of Car_2 are

$$\begin{aligned} & (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_2 \\ &= \{(\text{Shanghai}, 12 : 00, \emptyset), (\text{Suzhou}, 13 : 00, \{\text{Suzhou}\}), \\ & \quad (\text{Changzhou}, 14 : 30, \{\text{Suzhou}, \text{Changzhou}\}), \\ & \quad (\text{Nanjing}, 16 : 00, \{\text{Suzhou}, \text{Changzhou}, \text{Nanjing}\})\}. \end{aligned}$$

For the subsequent process of $\text{Car}_1 \sqcap \text{Car}_2$, we can compute:

$$\begin{aligned} & (\text{Shanghai}, 12 : 00, \emptyset) \circ (\text{Car}_1 \sqcap \text{Car}_2) \\ &= (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_1 \vee (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_2 \\ &= \{(\text{Shanghai}, 12 : 00, \emptyset), (\text{Suzhou}, 13 : 00, \{\text{Suzhou}\}), \\ & \quad (\text{Nanjing}, 16 : 00, \{\text{Suzhou}, \text{Nanjing}\})\}. \end{aligned}$$

The initial data state is $(\text{Shanghai}, 12 : 00, \emptyset)$, the final data states of $\text{Car}_1 \sqcap \text{Car}_2$ are acquired by taking the common space-time pair of the final data states of Car_1 and Car_2 and intersecting their messages. For $\text{Car}_1 \parallel \text{Car}_2$ the space-time pairs of the final data states of Car_1 and Car_2 are different. For example, the final data states of $\text{Car}_1 \parallel \text{Car}_2$ are:

$$\begin{aligned} & (\text{Shanghai}, 12 : 00, \emptyset) \circ (\text{Car}_1 \parallel \text{Car}_2) \\ &= (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_1 \wedge (\text{Shanghai}, 12 : 00, \emptyset) \circ \text{Car}_2 \\ &= \{(\text{Shanghai}, 12 : 00, \emptyset), (\text{Suzhou}, 13 : 00, \{\text{Suzhou}\}), \\ & \quad (\text{Wuxi}, 13 : 30, \{\text{Suzhou}, \text{Wuxi}\}), \\ & \quad (\text{Changzhou}, 14 : 30, \{\text{Suzhou}, \text{Changzhou}\}), \\ & \quad (\text{Nanjing}, 16 : 00, \{\text{Suzhou}, \text{Wuxi}, \text{Changzhou}, \text{Nanjing}\})\}. \end{aligned}$$

The initial data state is $(\text{Shanghai}, 12 : 00, \emptyset)$. The final data states of $\text{Car}_1 \parallel \text{Car}_2$ are acquired by taking the data states from both Car_1 and Car_2 .

IV. ALGEBRAIC REFINEMENT

In this section, we give the algebraic semantics of the STeC language based on the algebraic refinement relation. We first extend the definition of algebraic refinement $\sqsubseteq_A$ to STeC programs.

Definition 7 (Algebraic Refinement). *Given any process P , Q , $P \sqsubseteq_A Q$ iff $\forall (a, u, \sigma) \in ST. ((a, u, \sigma) \circ P \sqsubseteq_A (a, u, \sigma) \circ Q)$.*

The algebraic refinement relation between processes is given by the algebraic refinement relation between the testing of processes given any data state. The algebraic refinement equivalence is induced by algebraic refinement $\sqsubseteq_A$.

Definition 8 (Algebraic Equivalence). *$P =_A Q$ iff $P \sqsubseteq_A Q$ and $Q \sqsubseteq_A P$.*

Given a STeC program P , if the input of the testing of P is empty, then the output of it is empty.

A. Algebraic Refinement Laws

Some refinement laws can be deduced from the algebraic semantics of STeC.

Theorem 1. (1) $P \sqsubseteq_A \text{skip}@ (l, t); P$.

(2) $P \sqsubseteq_A P; \text{skip}@ (l, t)$.

(3) $P \parallel Q \sqsubseteq_A P$.

Proof. (1) For any (a, u, σ) , if $(a, u) \neq (l, t)$, then $(a, u, \sigma) \circ (\text{skip}@ (l, t); P) = \top \circ P = \top$, otherwise $(a, u, \sigma) \circ (\text{skip}@ (l, t); P) = (l, t, \sigma) \circ P = (a, u, \sigma) \circ P$. So $P \sqsubseteq_A \text{skip}@ (l, t); P$.

(2) For any (a, u, σ) , $(l_0, t_0, \sigma_0) \in (a, u, \sigma) \circ P$, if $(l_0, t_0) \neq (l, t)$, then $(l_0, t_0, \sigma_0) \circ \text{skip}@ (l, t) = \top$, otherwise $(l_0, t_0, \sigma_0) \circ \text{skip}@ (l, t) = \{(l_0, t_0, \sigma_0)\}$. Therefore $(a, u, \sigma) \circ (P; \text{skip}@ (l, t)) \subseteq (a, u, \sigma) \circ P$. Thus

$(a, u, \sigma)^\circ P \sqsubseteq_A (a, u, \sigma)^\circ (P; \text{skip}@ (l, t))$.
 (3) According to Definition 4, for any (a, u, σ) ,
 $(a, u, \sigma)^\circ P \wedge (a, u, \sigma)^\circ Q = (a, u, \sigma)^\circ P \cup (a, u, \sigma)^\circ Q \supseteq P$.
 So $P \parallel Q \sqsubseteq_A P$. $\square$

We notice that $P \sqsubseteq_A P \sqcup Q$ does not hold in this context. This violates the intuition that $P \vee Q$ looks like the ‘least upper bound’ of P and Q , but in fact it is not. Because of the way the algebraic refinement is defined and the properties of data state refinement, we can only prove that $(ST_\perp^*, \sqsubseteq_A)$ is a $\wedge$ semi-lattice.

B. An Example of Algebraic Refinement

Consider the second example in Section II-B, we analyse the algebraic refinement relation between Car_1 and $Car_1 \parallel Car_2$. For any $(a, u, \sigma) \in ST$, we know that

$$(a, u, \sigma)^\circ Car_1 \subset (a, u, \sigma)^\circ Car_1 \parallel Car_2,$$

so $(a, u, \sigma)^\circ Car_1 \parallel Car_2 \sqsubseteq_A (a, u, \sigma)^\circ Car_1$. So we can conclude $Car_1 \parallel Car_2 \sqsubseteq_A Car_1$.

V. DENOTATIONAL REFINEMENT

Based on the algebraic refinement between STeC programs, we establish the denotational semantics of the STeC language. We identify a STeC program as a binary relation $[P]$ between data states.

Definition 9 (Denotation). *The denotation of a STeC program P is a binary relation $[P]$ defined as follows:*

$$[P] =_{df} \{(s, t) \mid s^\circ P \sqsubseteq_A t\},$$

where $s, t \in ST$.

$\sqsubseteq_D$ describes the denotational refinement relationship between programs.

Definition 10 (Denotational Refinement). $P \sqsubseteq_D Q$ iff $\forall s, t \in ST \cdot (s^\circ Q \sqsubseteq_A t \Rightarrow s^\circ P \sqsubseteq_A t)$.

Lemma 1. $P \sqsubseteq_D Q$ iff $[Q] \subseteq [P]$.

Proof. Note that $s^\circ Q \sqsubseteq_A t$ iff $(s, t) \in [Q]$. $\square$

From the algebraic refinement relation between data states, we induce the denotational semantics of STeC programs. The first main result is that in this way, the denotational refinement is equivalent to the algebraic refinement.

Theorem 2. $\sqsubseteq_A = \sqsubseteq_D$.

Proof. For $\sqsubseteq_A \subseteq \sqsubseteq_D$, if $P \sqsubseteq_A Q$, by Definition 7 we have $s^\circ P \sqsubseteq_A s^\circ Q$ for any $s \in ST$. Thus for any $t \in ST$, $s^\circ Q \sqsubseteq_A t$ implies $s^\circ P \sqsubseteq_A s^\circ Q \sqsubseteq_A t$.

For $\sqsubseteq_D \subseteq \sqsubseteq_A$, suppose $P \sqsubseteq_D Q$, for any $s \in ST$, by Definition 5 we have $s^\circ Q = \begin{cases} \perp \\ T \end{cases}$. If $s^\circ Q = \perp$, which is $s^\circ Q \sqsubseteq_A \perp$, then for any $t \in ST$, $s^\circ Q \sqsubseteq_A t$. By assumption we know that $s^\circ P \sqsubseteq_A t$ (for any $t \in ST$). So according to Definition 2, we know $s^\circ P = \perp$. If $s^\circ Q = T$ where $T = \{t_1, t_2, \dots, t_n\} \in ST^*$, then for any $t_i \in T$

$(1 \leq i \leq n)$ we obtain $s^\circ Q \sqsubseteq_A t_i$, by assumption we acquire $s^\circ P \sqsubseteq_A t_i$. Because for any $t_i \in T$, $T = \bigwedge_{t_i \in T} t_i \sqsubseteq_A t_i$, so $s^\circ P \sqsubseteq_A T = s^\circ Q$. $\square$

A. An Example of Denotational Refinement

Considering the second example in Section II-B. Based on the data states of Car_1 and $Car_1 \parallel Car_2$ in Section III-B, the denotation of them can be obtained:

$$\begin{aligned} [Car_1] &= \{((Shanghai, 12 : 00, \emptyset), (Suzhou, 13 : 00, \{Suzhou\})), \\ &\quad ((Shanghai, 12 : 00, \emptyset), (Wuxi, 13 : 30, \{Suzhou, Wuxi\})), \\ &\quad ((Shanghai, 12 : 00, \emptyset), (Nanjing, 16 : 00, \\ &\quad \{Suzhou, Wuxi, Nanjing\}))\} \end{aligned}$$

$$\begin{aligned} [Car_1 \parallel Car_2] &= \{((Shanghai, 12 : 00, \emptyset), (Suzhou, 13 : 00, \{Suzhou, Wuxi\})), \\ &\quad ((Shanghai, 12 : 00, \emptyset), (Changzhou, 14 : 30, \\ &\quad \{Suzhou, Changzhou\})), ((Shanghai, 12 : 00, \emptyset), \\ &\quad (Nanjing, 16 : 00, \{Suzhou, Wuxi, Nanjing\})), \\ &\quad ((Shanghai, 12 : 00, \emptyset), (Nanjing, 16 : 00, \\ &\quad \{Suzhou, Changzhou, Nanjing\}))\} \end{aligned}$$

Since $[Car_1] \subset [Car_1 \parallel Car_2]$, we conclude that $Car_1 \parallel Car_2 \sqsubseteq_D Car_1$.

VI. OPERATIONAL REFINEMENT

In this section, we construct the operational refinement and semantics of STeC, based on the algebraic semantics in Section III-A and the algebraic refinement in Section IV.

A. Refinement Relations

We follows He’s machine state in [9] to identify the *machine state* as pairs:

$$(s, P)$$

where P is a STeC programming *text*, the rest of the program to be executed, $s \in ST$. When there is no STeC program to be executed, we use the empty word ϵ to represent it. The state:

$$(t, \epsilon)$$

is the final machine state of any execution of the STeC program. t represents the final data state of the STeC program. We give the operational step transition and the concept of divergence expressed by the step relation.

Definition 11 (Step Relation). *An operational step transition is a binary relation between machine states, denoted as $\rightarrow \subseteq (ST \times \mathbb{P}) \times (ST \times \mathbb{P})$. Define $(s, P) \rightarrow^1 (t, Q) =_{df} (s, P) \rightarrow (t, Q)$. Let $\rightarrow^n$ be the n -step transition relation: $(s, P) \rightarrow^n (t, Q) =_{df} \exists s', P' \cdot ((s, P) \rightarrow^{n-1} (s', P') \rightarrow^1 (t, Q))$ for any $n > 1$. Let $\rightarrow^*$ be the transitive closure of $\rightarrow$, that is, $(s, P) \rightarrow^* (t, Q) =_{df} \exists n \cdot ((s, P) \rightarrow^n (t, Q))$.*

The divergence gives the description of a STeC program which runs forever and never ends.

Definition 12 (Divergence). *The divergent STeC program performs as an infinite sequence of actions. It can be expressed as follows:*

$$(s^\circ P)^\dagger =_{df} \forall n \in \mathbb{N}, \exists t \in st, \exists Q \neq \epsilon \cdot ((s, P) \rightarrow^n (t, Q)),$$

where s is the initial data state, and Q represents the rest of the program to be executed.

Proposition 5. $((l, t, \sigma)^\circ Stop @ (l, t))^\dagger$.

A simple example of divergence in STeC is $Stop @ (l, t)$. $Stop @ (l, t)$ repeatedly proceeds action $Stop$ at location l and at time t . This means that if the space-time point (l, t) of $Stop @ (l, t)$ is matched with the given initial space-time point (a, u) , in other words $(l, t) = (a, u)$, we can represent that $((l, t, \sigma), Stop @ (l, t))^\dagger : ((l, t, \sigma), Stop @ (l, t)) \rightarrow^1 ((l, t, \sigma), Stop @ (l, t)) \rightarrow^1 \dots \rightarrow^1 ((l, t, \sigma), Stop @ (l, t))$.

In [9], He introduces the notion of consistency of the step relation linking the step relation to the algebraic semantics of programs. Here, we introduce the similar notion for the STeC language in the following way.

Definition 13 (Consistency of the step relation). *A step relation $\rightarrow$ is consistent with the algebraic semantics if*

$$s^\circ P =_A \begin{cases} \perp & \text{if } (s^\circ P)^\dagger \\ \{t \mid (s, P) \rightarrow^* (t, \epsilon)\} & \text{else} \end{cases}$$

The link between step relations and algebraic semantics of a STeC program is explored as above in two aspects:

1. A machine state (s, P) is divergent if and only if its testing $s^\circ P = \perp$.
2. If (s, P) is not divergent, the final data states which the testing of (s, P) can reach are exactly those the algebraic semantics has specified.

Next we give the definition of the operational refinement of the STeC language based on the step relation.

Definition 14. *An operational refinement is a binary relation between two STeC programs, the definition is given below:*

$$P \sqsubseteq_O Q =_{df} \left(\begin{array}{l} (s^\circ Q =_A \perp \Rightarrow s^\circ P =_A \perp) \wedge \\ \forall s \in ST \cdot \left(\forall t \cdot \left((s, Q) \rightarrow^* (t, \epsilon) \Rightarrow \left(\begin{array}{l} (s^\circ P =_A \perp) \vee \\ (s, P) \rightarrow^* (t, \epsilon) \end{array} \right) \right) \right) \end{array} \right),$$

where $\rightarrow$ is a consistent step relation.

B. Operational Semantics

The operational semantics attempts to interpret programs as transition systems with actions moving from one program state to another. Suppose that the space-time point of each STeC formula is matched with the initial space-time point, we give the operational semantics as followed.

- 1) $\frac{((a, u, \sigma), Stop @ (l, t)) \rightarrow^1 ((l, t, \sigma), Stop @ (l, t))}{(a, u) = (l, t)}$
- 2) $\frac{((a, u, \sigma), Skip @ (l, t)) \rightarrow^1 ((l, t, \sigma), \epsilon)}{(a, u) = (l, t)}$
- 3) $\frac{((a, u, \sigma), Send(m) @ (l, t)) \rightarrow^1 ((l, t, \sigma \cup \{m\}), \epsilon)}{(a, u) = (l, t)}$

- 4) $\frac{((a, u, \sigma), Get(m) @ (l, t)) \rightarrow^1 ((l, t, \sigma \setminus \{m\}), \epsilon)}{(a, u) = (l, t)}$
- 5) $\frac{((a, u, \sigma), Act(l', \delta) @ (l, t)) \rightarrow^1 ((l', t + \delta, \sigma), \epsilon)}{(a, u) = (l, t)}$
- 6) $\frac{((a, u, \sigma), Stat(\delta) @ (l, t)) \rightarrow^1 ((l, t + \delta, \sigma), \epsilon)}{(a, u) = (l, t)}$
- 7) $\frac{((a, u, \sigma), Sens(m, \delta) @ (l, t)) \rightarrow^1 ((l, t + \delta, \sigma \cup \{m\}), \epsilon)}{(a, u) = (l, t)}$
- 8) $\frac{((a, u, \sigma), Comp(m, m', \delta) @ (l, t)) \rightarrow^1 ((l, t + \delta, \sigma[m'/m]), \epsilon)}{(a, u) = (l, t)}$
- 9) $\frac{((l, t, \sigma), B \rightarrow P) \rightarrow^1 ((l(B), t + \tau(B), \sigma), P)}{\mathcal{T}(B) = true \wedge ((l, t, \sigma), B) \rightarrow^1 ((l(B), t + \tau(B), \sigma), \epsilon)}$
- 10) $\frac{((l, t, \sigma), B \rightarrow P) \rightarrow^1 ((l, t, \sigma), Stop @ (l, t))}{\mathcal{T}(B) = false}$
- 11) $\frac{((l, t, \sigma), P; Q) \rightarrow^n ((l', t', \sigma'), P'; Q)}{((l, t, \sigma), P) \rightarrow^n ((l', t', \sigma'), P')}$
- 12) $\frac{((l, t, \sigma), P \sqcap Q) \rightarrow^{\max(k, m)} ((l', t', \sigma' \cap \sigma''), E)}{((l, t, \sigma), P) \rightarrow^k ((l', t', \sigma'), E), ((l, t, \sigma), Q) \rightarrow^m ((l', t', \sigma''), E)}$
- 13) $\frac{((l, t, \sigma), P \parallel Q) \rightarrow^k ((l_1, t_1, \sigma_1), E) \mid ((l, t, \sigma), P \parallel Q) \rightarrow^m ((l_2, t_2, \sigma_2), E)}{((l, t, \sigma), P) \rightarrow^k ((l_1, t_1, \sigma_1), E) \mid ((l, t, \sigma), Q) \rightarrow^m ((l_2, t_2, \sigma_2), E)}$

Function $\max(m, n)$ returns the greater number of m, n . The operational semantics of STeC must be accordant with its algebraic semantics. As it can be seen in the last section, the link between the step relation and the algebraic refinement is built by consistent step relation. Thus the operational semantics of STeC also must be consistent step relations. We have the following theorem declaring this.

Theorem 3. *The operational semantics of STeC defined above is consistent.*

Proof. We prove it by the mathematical induction on the structure of the syntax of STeC.

Consider any STeC process P . If P is an atomic process. For $(a, u, \sigma)^\circ Stop @ (l, t)$, if $(a, u) \neq (l, t)$, there is no such operation rules for $Stop @ (l, t)$, so $(a, u, \sigma)^\circ Stop @ (l, t) =_A \perp$. If $(a, u) = (l, t)$, based on Proposition 5 it is easily seen that $((l, t, \sigma), Stop @ (l, t))^\dagger$. According to the algebraic semantics of STeC, we know that $(l, t, \sigma)^\circ Stop @ (l, t) =_A \perp$.

Other cases are similar. We only consider $(a, u, \sigma)^\circ P = (a, u, \sigma)^\circ Act(l', \delta) @ (l, t)$ here. If $(a, u) \neq (l, t)$, the condition is same for $Stop @ (l, t)$. Otherwise according to the algebraic semantics we get $(a, u, \sigma)^\circ Act(l', \delta) @ (l, t) = \{(l', t + \delta, \sigma)\}$. But this happens to be the same result from Rule 5 of the operational semantics. So $(a, u, \sigma)^\circ Act(l', \delta) @ (l, t) =_A \{t \mid ((a, u, \sigma), Act(l', \delta) @ (l, t)) \rightarrow^1 (t, \epsilon)\}$.

For the rest of the proof, we omit the discussion of the situation when $(a, u) \neq (l, t)$ since under which there is no operational rules.

Suppose that the operational semantics of any subprocesses of P is consistent.

If $P = B \rightarrow Q$, we consider two situations. If $\mathcal{T}(B) = false$, from the algebraic semantics, we can get $(l, t, \sigma)^\circ P = \perp$. From the operational semantics, we know that $((l, t, \sigma)^\circ P)^\dagger$. So $(l, t, \sigma)^\circ P =_A \perp$. If $\mathcal{T}(B) = true$, we

have $(l, t, \sigma)^\circ P = ((l, t, \sigma)^\circ B)^\circ Q$. Since for any $s \in ST$, by assumption there is: $s^\circ B =_A \{(l(B), t + \tau(B), \sigma)\}$ and $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$. According to the operational semantics of $B \rightarrow Q$ we have $(l, t, \sigma)^\circ P = ((l, t, \sigma)^\circ B)^\circ Q =_A (l(B), t + \tau(B), \sigma)^\circ Q =_A \{t \mid ((l(B), t + \tau(B), \sigma), Q) \rightarrow^* (t, \epsilon)\} =_A \{t \mid ((l, t, \sigma), P) \rightarrow^* (t, \epsilon)\}$. So $(l, t, \sigma)^\circ P =_A \{t \mid ((l, t, \sigma), P) \rightarrow^* (t, \epsilon)\}$.

If $P = Q; R$, based on the algebraic semantics of STeC we acquire $(l, t, \sigma)^\circ P = ((l, t, \sigma)^\circ Q)^\circ R$. By the assumption the operational semantics of Q or R is consistent, so $s^\circ R =_A \{t \mid (s, R) \rightarrow^* (t, \epsilon)\}$ and $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$ hold for any $s \in ST$. Therefore according to the operational semantics of $Q; R$, there is $(l, t, \sigma)^\circ P = ((l, t, \sigma)^\circ Q)^\circ R =_A \{t \mid ((l, t, \sigma), Q) \rightarrow^* (t, \epsilon)\}^\circ R =_A \{t \mid ((l, t, \sigma), Q) \rightarrow^* ((l', t', \sigma'), \epsilon) \wedge ((l', t', \sigma'), R) \rightarrow^* (t, \epsilon)\} =_A \{t \mid ((l, t, \sigma), Q; R) \rightarrow^* ((a', u', \sigma'), R) \rightarrow^* (t, \epsilon)\} =_A \{t \mid ((l, t, \sigma), P) \rightarrow^* (t, \epsilon)\}$.

If $P = Q \sqcap R$, then by the algebraic semantics of STeC, there is $(l, t, \sigma)^\circ P = (l, t, \sigma)^\circ Q \vee (l, t, \sigma)^\circ R$. Since the operational semantics of Q and R are consistent, we have $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$ and $s^\circ R =_A \{t \mid (s, R) \rightarrow^* (t, \epsilon)\}$ hold for any $s \in ST$. So $(l, t, \sigma)^\circ P =_A \{(l', t', \sigma' \cap \sigma'') \mid ((l, t, \sigma), Q) \rightarrow^* ((l', t', \sigma'), \epsilon) \wedge ((l, t, \sigma), R) \rightarrow^* ((l', t', \sigma''), \epsilon)\}$. From the operational semantics of $Q \sqcap R$, there is $(l, t, \sigma)^\circ P =_A \{t \mid ((l, t, \sigma), Q \sqcap R) \rightarrow^* (t, \epsilon)\}$.

If $P = Q \parallel R$, there is $(l, t, \sigma)^\circ P = (l, t, \sigma)^\circ Q \wedge (l, t, \sigma)^\circ R$ from the algebraic semantics. Because the operational semantics of Q and R are consistent, we have $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$ and $s^\circ P =_A \{t \mid (s, P) \rightarrow^* (t, \epsilon)\}$ hold for any $s \in ST$. Thus $(l, t, \sigma)^\circ P = (l, t, \sigma)^\circ Q \cup (l, t, \sigma)^\circ R =_A \{t \mid ((l, t, \sigma), Q) \rightarrow^* (t, \epsilon) \vee ((l, t, \sigma), P) \rightarrow^* (t, \epsilon)\}$. Based on the operational semantics of $Q \parallel R$, it is easy to get $(l, t, \sigma)^\circ P =_A \{t \mid ((l, t, \sigma), Q \parallel R) \rightarrow^* (t, \epsilon)\}$. $\square$

Finally in this section, the relationship between the operational refinement and the algebraic refinement of STeC is discussed. We have seen that we can introduce the operational semantics of a STeC program from its algebraic semantics. The next theorem declares that the operational refinement are equivalent to the algebraic refinement.

Theorem 4. $\sqsubseteq_A = \sqsubseteq_O$.

Proof. For $\sqsubseteq_A \subseteq \sqsubseteq_O$. Suppose $P \sqsubseteq_A Q$. For any $s \in ST$, since the operational step relation $\rightarrow$ of STeC is consistent, we know that $s^\circ Q =_A \begin{cases} \perp & \text{if } (s^\circ Q)^\dagger \\ \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\} & \text{else} \end{cases}$. If $s^\circ Q =_A \perp$, since $s^\circ P \sqsubseteq_A s^\circ Q$, $s^\circ P =_A \perp$. If $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$, because of $s^\circ Q \subseteq s^\circ P$, for any $t \in s^\circ Q$, we know that $t \in s^\circ P$, thus we get $s^\circ P =_A \perp$ or $(s, P) \rightarrow^* (t, \epsilon)$. For $\sqsubseteq_O \subseteq \sqsubseteq_A$. Suppose $P \sqsubseteq_O Q$. For any $s \in ST$, from the consistency of $\rightarrow$ we have $s^\circ Q =_A \begin{cases} \perp & \text{if } (s^\circ Q)^\dagger \\ \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\} & \text{else} \end{cases}$. If $s^\circ Q =_A \perp$, from Definition 14 we acquire $s^\circ P =_A \perp$. If $s^\circ Q =_A \{t \mid (s, Q) \rightarrow^* (t, \epsilon)\}$, we know from Definition 14 that for any $t \in s^\circ Q$, either $s^\circ P =_A \perp$ or $(s, P) \rightarrow^* (t, \epsilon)$. In both situations $s^\circ P \sqsubseteq_A s^\circ Q$ holds. $\square$

C. An Example of Operational Refinement

Again, let us consider the second example in Section II-B. According to operational semantics, we have

$$\begin{aligned} & (Shanghai, 12 : 00, \emptyset)^\circ Car_1 \\ =_A & \{(Suzhou, 13 : 00, \{Suzhou\}), (Wuxi, 13 : 30, \{Suzhou, Wuxi\}), \\ & (Nanjing, 16 : 00, \{Suzhou, Wuxi, Nanjing\})\}. \\ & (Shanghai, 12 : 00, \emptyset)^\circ Car_1 \parallel Car_2 \\ =_A & \{(Suzhou, 13 : 00, \{Suzhou\}), (Wuxi, 13 : 30, \{Suzhou, Wuxi\}), \\ & (Changzhou, 14 : 30, \{Suzhou, Changzhou\}), \\ & (Nanjing, 16 : 00, \{Suzhou, Wuxi, Nanjing\})\}, \\ & (Nanjing, 16 : 00, \{Suzhou, Changzhou, Wuxi, Nanjing\})\}. \end{aligned}$$

From this we can know that if $(a, u, \sigma)^\circ Car_1 =_A \perp$, which is to mean $(a, u) \neq (Shanghai, 12 : 00)$, then $(a, u, \sigma)^\circ Car_1 \parallel Car_2 = \perp$. If $(a, u) = (Shanghai, 12 : 00)$, from the operational results above we can see that if $((a, u, \sigma), Car_1) \rightarrow^* (t, \epsilon)$ for some $t \in (Shanghai, 12 : 00, \emptyset)^\circ Car_1$, then we have $((a, u, \sigma), Car_1 \parallel Car_2) \rightarrow^* (t, \epsilon)$. So $Car_1 \parallel Car_2 \sqsubseteq_O Car_1$.

VII. MAIN THEOREM

From the previous sections, it has already become apparent that we have built the denotational and operational refinement relations of STeC based on its algebraic refinement relations following He's method. According to these refinement relations, we construct the operational semantics of STeC which is stated informally in Section II-A. The crucial idea of He's method is of by starting from algebraic properties of a language, we can develop the whole three semantics of the general language and prove the equivalence of them, which is also called the full abstraction property. In this section, we give the main result of this paper stated as following theorem, which declares the full abstraction of STeC language.

Theorem 5. $\sqsubseteq_A = \sqsubseteq_D = \sqsubseteq_O$.

Proof. A direct result from Theorem 2, 4. $\square$

VIII. CONCLUSION

This paper mainly proposes a new theory of building the semantics of the formal specification language – STeC. This idea originally comes from recent He's work [9] about linking theories of programming with a new theoretical approach. The main feature of this new technique is that the whole unifying theories of formal languages can be established based on the primary, intuitive algebraic properties of computer programs. In this paper, we first introduce the basic algebraic properties of the STeC language, including the algebraic refinement relations between data states. Then we deduce the algebraic and denotational semantics, and finally the operational semantics from the axioms and the rules of these basic algebraic properties. The algebraic refinement relations between data states is the crucial link between two other semantics. The main result of this paper is that we prove the consistency of three formalisms, which in turn justifies the rationality of using this method. However this paper has many weaknesses. For

example we can not give any relations between P and $P \sqcap Q$. Maybe $P \sqsubseteq_A P \sqcap Q$ should be true in STeC because for requirements, since the most deterministic data should come from the common part of $P \sqcap Q$.

As future work, we will probably focus on exploring different refinement properties between processes using different refinement orders. It is well-known that there are three kinds of non-deterministic refinement relations between data sets in the power domain theory, called Hoare, Plotkin and Smyth refinement relations in general. These refinement relations are defined based on the refinement of data. We take the Smyth refinement relation $\sqsubseteq_S$ [16] as an example. For two data sets M and N , we defines $M \sqsubseteq_S N$ if, and only if $\forall n \in N, \exists m \in M \bullet m \sqsubseteq n$. It is obvious that if N is a subset of M then $M \sqsubseteq_S N$. Thus, we have that the algebraic refinement $\sqsubseteq_A$ introduced in this paper is a special kind of Smyth refinement relation. The classical non-deterministic behaviour in the framework of UTP and the STeC language should be investigated. We are also aiming at finding meaningful usages of our theory both in theoretical and applicative aspects.

ACKNOWLEDGE

This work is supported by the National Natural Science Foundation of China (No.61370100), and the Shanghai Knowledge Service Platform Project (No.ZF1213) — a project supported by the Shanghai Committee of Science and Technology (No.14511100400).

REFERENCES

- [1] Y. Chen, “Stec: A location-triggered specification language for real-time systems,” in *Object/Component/Service-Oriented Real-Time Distributed Computing Workshops (ISORCW)*, 2012 15th IEEE International Symposium on. IEEE, 2012, pp. 1–6.
- [2] C. A. R. Hoare and P. E. Lauer, “Consistent and complementary formal theories of the semantics of programming languages,” *Acta Informatica*, vol. 3, no. 2, pp. 135–153, 1974.
- [3] C. A. R. Hoare and H. Jifeng, *Unifying theories of programming*. Prentice Hall Englewood Cliffs, 1998, vol. 14.
- [4] C. A. R. Hoare, “Algebra and models,” in *Program Design Calculi*. Springer, 1993, pp. 161–195.
- [5] S. Seres, J. M. Spivey, and C. Hoare, “Algebra of logic programming,” in *ICLP*, 1999, pp. 184–199.
- [6] R.-J. Back, “A calculus of refinements for program derivations,” *Acta Informatica*, vol. 25, no. 6, pp. 593–624, 1988.
- [7] C. A. R. Hoare, *Communicating sequential processes*. Springer, 1978.
- [8] B. Roscoe, *The theory and practice of concurrency*. Prentice Hall, 1997.
- [9] H. Jifeng, “A new roadmap on linking theories of programming,” 2016, an invited talk on UTP 2016.
- [10] G. M. Reed and A. W. Roscoe, “A timed model for communicating sequential processes,” *Theoretical Computer Science*, vol. 58, no. 1, pp. 249–261, 1988.
- [11] F. Moller and C. Tofts, *A temporal calculus of communicating systems*. Springer, 1990.
- [12] H. Wu, Y. Chen, and M. Zhang, “On denotational semantics of spatial-temporal consistency language–stec,” in *Theoretical Aspects of Software Engineering (TASE)*, 2013 International Symposium on. IEEE, 2013, pp. 113–120.
- [13] Y. Zhang, F. Mallet, and Y. Chen, “Timed automata semantics of spatial-temporal consistency language stec,” in *2014 Theoretical Aspects of Software Engineering Conference (TASE)*. IEEE, 2014, pp. 201–208.
- [14] Z. Yang, B. Xiao, and Y. Chen, “Modeling and verification of space-air-ground integrated networks on requirement level using stec,” in *Theoretical Aspects of Software Engineering (TASE)*, 2015 International Symposium on. IEEE, 2015, pp. 131–134.
- [15] H. Kangli, C. Yixiang, Z. Min, and Z. Yuanrui, “Pstec: A location-time driven modelling formalism for probabilistic real-time systems,” 2016, to appear on 18th International GI/ITG Conference on MME and DFT.
- [16] M. B. Smyth, “Power domains,” *Journal of Computer and System Sciences*, vol. 16, no. 1, pp. 23–36, 1978.